

MYD-LD25X Linux 软件开发指南



文件状态： [] 草稿 [√] 正式发布	文件编号：	MYIR-MYD-LD25X-SW-DG-ZH-L6.6.48
	文件版本：	V1.2[文档]
	作 者：	MSW0192
	发布日期：	2024-09-11
	最近更新：	2025-02-17

版本历史

版本	作者	审核者	日期	备注
V1.0[文档]	MSW0192	MSW0041	2024-09-11	正式版本发布
V1.1[文档]	MSW0192	MSW0041	2024-11-08	适配 MYD-LD257-8E1D 型号
V1.2[文档]	MSW0192	MSW0041	2025-02-17	升级 6.6.48 版本



目 录

版本历史	- 2 -
目 录	- 3 -
1. 系统开发概述	- 5 -
1.1. 软件资源	- 6 -
2. 如何搭建开发环境	- 7 -
2.1. 开发主机环境	- 7 -
2.2. 一键安装交叉编译工具链	- 10 -
3. 如何烧录系统镜像	- 12 -
4. 板级支持包的单独编译和更新	- 13 -
4.1. 板级支持包介绍	- 13 -
4.2. 板载 BootLoader 编译与更新	- 14 -
4.3. 板载 Kernel 编译与更新	- 18 -
5. 搭建基本的 Yocto 环境	- 22 -
5.1. 简介	- 22 -
5.2. 获取源码	- 23 -
5.3. 快速编译开发板镜像	- 24 -
5.4. 构建交叉编译链	- 28 -
6. 如何适配底板硬件	- 29 -
6.1. meta-bsp 层介绍	- 29 -
6.2. 如何创建自己的 machine	- 30 -
6.3. 如何创建您的设备树	- 34 -
6.4. 如何根据您的硬件配置 CPU 功能管脚	- 37 -
6.5. 如何使用自己配置的管脚	- 39 -
7. 应用的开发与启动	- 40 -



7.1. 基于 Makefile 的应用	- 40 -
7.2. 基于 Qt 的应用	- 44 -
7.3. 应用程序开机自启动	- 45 -
8. 参考资料	- 49 -
附录一 联系我们	- 50 -
附录二 售后服务与技术支持	- 52 -



1. 系统开发概述

嵌入式 Linux 系统是一个比较庞大软件系统，包括用于片内外设的板级软件包 BSP (Board Support Package)，狭义的理解可认为是系统引导程序，驱动程序等；Linux 系统内核，它包括众多协议栈，资源管理和调度器等；Rootfs 根文件系统，应用的运行环境，初始化脚本和服务资源等。如何将这些资源整合成一个整体并且可以被很好地管理起来，就需要一个框架来管理。

嵌入式 Linux 系统软件有许多开源的构建框架，目前比较常见的有 Buildroot, Yocto, Busybox 等等。其中 Yocto 使用强大的资源整合手段和高定制化的架构，可以很方便地构建出适合嵌入式产品的 Linux 系统。同时 Yocto 不仅是一个制作文件系统工具，同时提供整套的基于 Linux 的开发和维护工作流程，使底层嵌入式开发者和上层应用开发者在统一的框架下开发，解决了传统开发方式下零散和无管理的开发形态。

MYD-LD25X 系列开发平台上，使用 Yocto 部署开发环境、构建系统、Linux 应用程序集成、镜像的更新等。系统开发人员熟悉第五章的 Yocto 的开发流程之后，就可以参照第六章的如何适配底板硬件针对实际项目需求对 BSP 进行差异化的定制，就可以很快地将系统移植到基于 MYD-LD25X 核心板设计的硬件平台之上。

注：本文档并不包含 Yocto 项目以及 Linux 系统相关基础知识的介绍，适合有一定开发经验的嵌入式 Linux 系统开发人员。针对用户在进行二次开发过程中可能会使用到的一些具体功能，我们也提供了详细的应用开发手册。



1.1. 软件资源

MYD-LD25X 搭载基于 Linux 6.6.48 版本内核的操作系统，提供了丰富的系统资源和其他软件资源。开发板出厂附带嵌入式 Linux 系统开发所需要的交叉编译工具链，ATF 源代码，U-Boot 源代码，Linux 内核和各驱动模块的源代码，以及适用于 Windows 桌面环境和 Linux 桌面环境的各种开发调试工具，应用开发样例等。具体包含的软件信息请参考《MYD-LD25X SDK 发布说明-V1.2》中第 2 章软件信息中的说明。



2. 如何搭建开发环境

本章主要介绍基于 MYD-LD25X 开发板进行环境搭建，包括整个开发流程所需的一些软硬件环境，包括必要的硬件配件，软件工具等，具体的准备工作下面将进行详细介绍。

2.1. 开发主机环境

1) 主机软硬件要求

- 主机硬件

Yocto 项目的构建对开发主机的要求比较高，要求处理器具有双核以上 CPU，8GB 以上内存，400GB 硬盘或更高配置。可以是安装 Linux 系统的主机，也可以是运行 Linux 系统的虚拟机。

- 主机操作系统

构建 Yocto 项目的主机操作系统可以有很多种选择。我们这里选择的是 **Ubuntu20.04 64bit** 版本，后续开发也是以此系统为例进行介绍。

2) 主机环境配置

当安装完 ubuntu 20.04 64bit 系统后，可以先进行适当的配置，为后续开发做准备。

- 设置 root 密码

开发操作请尽可能使用**普通用户**权限，也就是主机名为自己的用户名而非 root，避免在开发过程中出现由于权限问题导致的问题。

```
$ sudo passwd root
```

- 更换清华源

Ubuntu 安装工具常用命令就是 apt-get，但是安装完系统后，默认的源为国外服务器，下载会很慢，所以需要更换为清华源。打开清华大学开源软件镜像站：<https://mirrors.tuna.tsinghua.edu.cn/help/ubuntu/>，选择你的 ubuntu 版本，如 20.04，Ubuntu 的软件源配置文件是/etc/apt/sources.list。将系统自带的该文件做个备份，将该文件替换为下面内容，即可使用 TUNA 的软件源镜像。这个不是必须更新，但是更新后安装包会比较快速。



```
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-updates main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-updates main restricted universe multiverse
deb https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-backports main restricted universe multiverse
# deb-src https://mirrors.tuna.tsinghua.edu.cn/ubuntu/ focal-backports main restricted universe multiverse

deb http://security.ubuntu.com/ubuntu/ focal-security main restricted universe multiverse
# deb-src http://security.ubuntu.com/ubuntu/ focal-security main restricted universe multiverse
```

● 安装 ssh

安装完 ssh 服务后，可以在 window 环境下，用远程串口调试工具可以通过 ssh2 的方式连接到 ubuntu 进行后续开发，串口调试工具可以通过网络搜索教程学习。

```
$ sudo apt-get install openssh-server
```

给用户生成密钥：

```
$ su user
$ ssh-keygen -t rsa
```

● 配置 samba

samba 可以直接在 window 下以文件夹形式访问 ubuntu 的内容，读写更方便。输入以下命令安装 samba：

```
$ apt-get install samba
```

在/etc/samba/smb.conf 中加入用户配置，以下配置以用户名“myir”为例，实际配置请根据具体情况配置自己的用户名，如下配置可以直接添加到配置文件末尾：

```
[myir]
path = /home/myir
valid users = myir
browseable = yes
public = yes
writable = yes
```

创建账号并设置密码：

```
$ sudo smbpasswd -a myir
New SMB password:
Retype new SMB password:
Added user myir.
```

/etc/init.d/smbd restart 重启 samba 服务：

```
$ /etc/init.d/smbd restart
```

- 0 -



```
[ ok ] Restarting smbd (via systemctl): smbd.service.
```

- 配置 git

```
$ git config --global user.name "user"
$ git config --global user.email "email"
$ git config --list
```

- 安装必要工具

本小节非常重要，请确保主机环境正确执行了以下操作，并且在完成如下操作后重启。

```
$ sudo apt-get update
$ sudo apt-get install -y gawk wget git diffstat \
unzip texinfo gcc-multilib build-essential chrpath socat libssl1.2-dev \
xterm sed cvs subversion coreutils texi2html docbook-utils python-pysqlite2 \
help2man make gcc g++ desktop-file-utils libgl1-mesa-dev libglu1-mesa-dev \
mercurial autoconf automake groff curl lzop asciidoc u-boot-tools cpio \
sudo locales bc libncurses5-dev screen flex bison vim-tiny \
device-tree-compiler xvfb libgtk2.0-dev libssl-dev net-tools libyaml-dev \
rsync liblz4-tool zstd python3-pip git-lfs iputils-ping jq
$ sudo rm -rf /var/lib/apt/lists/*
$ sudo curl https://mirrors.tuna.tsinghua.edu.cn/git/git-repo > /usr/bin/repo
$ sudo chmod a+x /usr/bin/repo
$ sudo ln -s /usr/bin/python3 /usr/bin/python
$ sudo sed -i -e 's/# en_US.UTF-8 UTF-8/en_US.UTF-8 UTF-8/' /etc/locale.gen
$ echo 'LANG="en_US.UTF-8"' | sudo tee /etc/default/locale > /dev/null
$ sudo dpkg-reconfigure --frontend=noninteractive locales
$ sudo update-locale LANG=en_US.UTF-8
$ sudo pip3 install pyusb usb crypto ecdsa crcmod tqdm pycryptodome pycryptodomex
pyelftools
```



2.2. 一键安装交叉编译工具链

我们在使用 Yocto 构建完系统镜像之后，还可以使用 Yocto 构建一套可扩展的 SDK。在米尔提供的资料中包含编译好的 SDK 包，位于：03-Tools/Compile Toolchain/，SDK 文件功能描述如下表：

表 2-1. 编译工具链

工具链文件名	描述
myir-image-full-openstlinux-weston-myd-ld25x.rootfs-x86_64-toolchain-5.0.3-snapshot.sh	包含一个独立的交叉开发工具链还提供 qmake，目标平台的 sysroot，Qt 应用开发所依赖的库和头文件等。用户可以直接使用这个 SDK 来建立一个独立的开发环境

这里先介绍 SDK 的安装步骤，如下：

- 拷贝 SDK 到 Linux 目录

将 SDK 安装脚本文件拷贝到 Ubuntu 下的用户工作目录，，如下：

```
$ mkdir myd-ld25x-toolchain;cd ~/myd-ld25x-toolchain
$ cp ~/03-Tools/Compile Toolchain/* ./
$ ls ~/myd-ld25x-toolchain
myir-image-full-openstlinux-weston-myd-ld25x.rootfs-x86_64-toolchain-5.0.3-snapshot.sh
```

- 执行安装脚本

以普通用户权限执行shell 脚本，运行中会提示安装路径，默认在/opt目录下。本例程把 qt 工具链安装在/home/myir/myd-ld25x-toolchain /目录，如下：

```
$ ./myir-image-full-openstlinux-weston-myd-ld25x.rootfs-x86_64-toolchain-5.0.3-snapshot.sh
ST OpenSTLinux - Weston - (A Yocto Project Based Distro) SDK installer version 5.0.3-snapshot
=====
=====
Enter target directory for SDK (default: /opt/st/myd-ld25x/5.0.3-snapshot): ./
You are about to install the SDK to "/home/myir/myd-ld25x-toolchain". Proceed [Y/n]? y
Extracting
SDK.....
.....
.....
```



```
.....done
```

```
Setting it up...done
```

```
SDK has been successfully set up and is ready to be used.
```

```
Each time you wish to use the SDK in a new shell session, you need to source the environment setup script e.g.
```

初始化环境变量：

```
$ source ~/myd-ld25x-toolchain/environment-setup-cortexa35-ostl-linux
```

● 测试 SDK

使用\$CC 或者\$CXX 来测试交叉编译的 gcc 和 g++是否被安装完成：

```
$CC -v
```

```
Using built-in specs.
```

```
COLLECT_GCC=aarch64-ostl-linux-gcc
```

```
COLLECT_LTO_WRAPPER=/media/home/beste/03_toolchain/lou-ld25x/sysroots/x86_64-ostl_sdk-linux/usr/libexec/aarch64-ostl-linux/gcc/aarch64-ostl-linux/13.3.0/lto-wrapper
```

```
Target: aarch64-ostl-linux
```

```
.....
```

```
Supported LTO compression algorithms: zlib zstd
```

```
gcc version 13.3.0 (GCC)
```

注意：初始化环境变量的操作仅会应用到当前开启的终端窗口中，不论以什么方式新开启的窗口，默认都没有加载 ld25x 的 SDK 交叉编译链，所以，如果需要进行 Yocto 构建或者其他的开发内容，请新开启一个窗口，避免交叉编译链导致的环境出错。



3. 如何烧录系统镜像

米尔公司设计的 MYD-LD25X 系列开发板是基于 ST 公司 的 STM32MP257 系列微处理器，其启动方式多样，所以需要不同的更新工具与方法。用户可以根据需求选择不同的方式进行更新。更新方式主要有以下几种：

- 通过 STM32CubeProgrammer：是一种 Windows 软件，可以通过 typeC 直接连接开发板进行烧录，无需其他存储介质。
- 制作 TF 卡启动器：适用于研发调试，快速启动等场景，启动器并不能烧录镜像。
- 制作 TF 卡烧录器：适用于批量生产烧写 emmc。

如需了解更详细的镜像烧录方法，请查阅《MYD-LD25X 量产指导手册-V1.2》，该文档内部包含了详细的 MYD-LD25X 各型号的，每种镜像的不同烧录方式的准备步骤和操作说明。



4. 板级支持包的单独编译和更新

4.1. 板级支持包介绍

为了适配用户新的硬件平台，首先需要了解米尔的 MYD-LD25X 开发板提供了哪些资源，具体的信息可以查看《MYD-LD25X SDK 发布说明-V1.2》。除此之外我们也对 BSP 各个部分中需要改动的一些文件列表整理出来，方便用户查找和修改，具体内容如下表所示：

表 4-1. 板级支持包配置信息

项目	设备树	说明
U-Boot	arch/arm/dts/myb-stm32mp257x-2GB.dts	2GB RAM 的 U-Boot 设备树定义文件
	arch/arm/dts/myb-stm32mp257x-2GB-resmem.dtsi	2GB RAM 的保留内存设备树配置包含文件
	arch/arm/dts/myb-stm32mp257x-2GB-u-boot.dtsi	U-Boot 特定设备树配置包含文件
	arch/arm/dts/myb-stm32mp257x-1GB.dts	1GB RAM 的 U-Boot 设备树定义文件
	arch/arm/dts/myb-stm32mp257x-1GB-resmem.dtsi	1GB RAM 的保留内存设备树配置包含文件
	arch/arm/dts/myb-stm32mp257x-1GB-u-boot.dtsi	U-Boot 特定设备树配置包含文件
	arch/arm/dts/myb-stm32mp257x-base.dtsi	基础设备树配置包含文件
	configs/myd_ld25x_2G_defconfig	2G ddr U-Boot 配置文件
	configs/myd_ld25x_1G_defconfig	1G ddr U-Boot 配置文件
Kernel	arch/arm64/configs/myd_stm32mp257x_defconfig	默认内核配置文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-2GB.dts	2GB RAM 的设备树定义文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-2GB-ethswitch.dts	2GB RAM 的以太网交换机的设备树定义文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-2GB-resmem.dtsi	2GB RAM 的保留内存设备树包含文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-1GB.dts	1GB RAM 的设备树定义文件



arch/arm64/boot/dts/myir/myb-stm32mp257x-1GB-ethswitch.dts	1GB RAM 的以太网交换机的设备树定义文件
arch/arm64/boot/dts/myir/myb-stm32mp257x-1GB-resmem.dtsi	1GB RAM 的保留内存设备树包含文件
arch/arm64/boot/dts/myir/myb-stm32mp257x-base.dtsi	基础设备树配置包含文件
arch/arm64/boot/dts/myir/myb-stm32mp257x-ethswitch.dtsi	以太网交换机设备树配置包含文件

下面章节重点讲述用户基于我们提供的 Bootloader, Kernel 和 Yocto 源代码和数据进行的流程。

4.2. 板载 BootLoader 编译与更新

U-boot 是一个功能非常丰富的开源启动引导程序，包括内核引导，下载更新等众多方面，在嵌入式领域应用十分广泛，可查看官网获取更多信息 <http://www.denx.de/wiki/U-Boot/WebHome>

4.2.1. 获取 uboot 源码

获取 uboot 源码可以通过以下两种方式获取，任选其一即可。

1) 从 github 获取源码

创建目录用于存放 uboot，然后按照如下命令将 github 中的 uboot 源码获取到本地。

```
$ mkdir bsp;cd bsp
$ git clone https://github.com/MYiR-Dev/myir-st-u-boot.git \
-b develop-ld25x-v2023.10
```

2) 从光盘镜像获取源码

解压光盘镜像后进入 04-Sources，将源码压缩包 MYD-LD25X-Distribution-L6.6.48-V1.0.0.tar 解压到主机中使用，进入到解压后的 MYD-LD25X-Uboot-L2023.10-V1.0.0 目录就是 uboot 源码了。

```
$ mkdir MYD-LD25X
$ tar xf 04-Sources/MYD-LD25X-Distribution-L6.6.48-V1.0.0.tar\
-C MYD-LD25X
$cd MYD-LD25X/MYD-LD25X-Distribution-L6.6.48-V1.0.0/MYD-LD25X-Uboot-L2023.10-
V1.0.0
```



4.2.2. 编译 Bootloader

查看 MYD-LD25X-Uboot-L2023.10-V1.0.0 的目录结构：

```
$ ls -la
build/                编译生成的文件目录
build-uboot-en.sh     英文构建脚本
build-uboot-zh.sh     中文构建脚本
deploy/              部署用的文件目录
FIP_artifacts/        FIP 相关文件
fiptool-stm32mp       STM32MP FIP 工具
Makefile.sdk SDK     编译规则文件
myir-st-external-dt/  U-Boot 的外部设备树文件
myir-st-optee/        OP-TEE 文件目录
myir-st-u-boot/       U-Boot 源码目录
output/              fip 最终文件输出目录
README_en            README 英文版本
README_zh            README 中文版本
```

运行一键编译脚本，将会自动编译。以下选择 2GB 型号为例，如需编译 1GB 型号请在提示中自行选择。

```
$ ./build-uboot-zh.sh
-----脚本开始运行-----

编译链正常
请选择需要构建的 ddr 大小配置(1/2):
1 - 1GB 型号 适用于 MYD-LD257-8E1D
2 - 2GB 型号 适用于 MYD-LD257-8E2D
请输入选择 (1 或 2): 2
正在运行 compile_2GB...
1GB 构建开始
```



```
*****
* 开始配置 Uboot. *
*****
[ ] 100%
*****
* 开始编译 Uboot. *
*****
[ ] 100%
*****
* 开始配置 Optee. *
*****
[ ] 100%
*****
* 开始构建 Fip 文件. *
*****
[ ] 100%
*****
**
* 编译完成，fip 文件位于 /home/myir/MYD-LD25X/MYD-LD25X-Uboot-L2023.10-
V1.0.0/Bootloader/output *
*****
*
```

最后编译完成的文件会保存在和脚本同级目录的 output 中，内容如下：

```
$ ls output/ -l
fip-myb-stm32mp257x-2GB-optee-emmc.bin
fip-myb-stm32mp257x-2GB-optee-sdcard.bin
```

4.2.3. 更新 Bootloader

本章节描述的文件单独烧录方式为开发板上使用命令烧录。

1) 烧录 fip.bin 引导固件镜像包到开发板

注：以下以 2GB 的 fip 文件为例，如果开发板为 MYD-LD257-8E1D 请自行将下面操作中的命令里的文件替换为 1GB 对应的文件。



- 烧录 fip-myb-stm32mp257x-2GB-optee-emmc.bin 到开发板

将编译好的 fip-myb-stm32mp257x-2GB-optee-emmc.bin 拷贝到 eMMC 开发板，可以通过 scp 或者存储介质如 U 盘等传输到开发板，这里以 scp 为例，在开发板串口调试串口执行：

```
# scp beste@192.168.40.21:~/scpfip/fip-myb-stm32mp257x-2GB-optee-emmc.bin /home/root
# dd if=fip-myb-stm32mp257x-2GB-optee-emmc.bin \
of=/dev/mmcbk1p3 bs=1M conv=fsync
# dd if=fip-myb-stm32mp257x-2GB-optee-emmc.bin
of=/dev/mmcbk1p4 bs=1M conv=fsync
```

完全更新完成还需要将开发板中原本的 env 清除。断电后重启，进入 Uboot 命令行，需要注意的是，由于 Uboot 的等待 delay 时间默认改为 0s，重启后并在注意到如下图所示的 Uboot 版本信息提示时按住回车 Enter 键，在进入 Uboot 命令行后松开，方可成功进入 Uboot 命令行：

```
I/TC: Primary CPU switching to normal world boot
INFO: BL31: Preparing for EL3 exit to normal world
INFO: Entry point address = 0x84000000
INFO: SPSR = 0x3c5
I/TC: Reserved shared memory is disabled
I/TC: Dynamic shared memory is enabled
I/TC: Normal world virtualization support is disabled
I/TC: Asynchronous notifications are enabled

U-Boot 2023.10-stm32mp-r1 (Feb 27 2025 - 02:22:11 +0000)

CPU: STM32MP257DAK Rev.Y
Model: MYIR STM32MP257F Evaluation Board
Board: stm32mp2 (myir,myb-stm32mp257x-2GB)
DRAM: 2 GiB
optee optee: OP-TEE: revision 4.0 (ca6f6578)
I/TC: Reserved shared memory is disabled
I/TC: Dynamic shared memory is enabled
I/TC: Normal world virtualization support is disabled
I/TC: Asynchronous notifications are enabled
Core: 431 devices, 40 uclasses, devicetree: board
WDT: Started watchdog with servicing every 1000ms (32s timeout)
NAND: 0 MiB
MMC: STM32 SD/MMC: 0, STM32 SD/MMC: 1
Loading Environment from MMC... OK
In: serial
Out: serial
Err: serial
invalid MAC address 2 in OTP 00:00:00:00:00:00
Net: eth1: eth1@482c0000, eth0: eth2@482d0000
No EFI system partition
No EFI system partition
Failed to persist EFI variables
Hit any key to stop autoboot: 0
STM32MP>
STM32MP>
STM32MP>
```

图 4-1. Uboot 命令行

进入命令行后输入一些内容即可完成 env 的擦除：

```
STM32MP> env erase
Erasing Environment on MMC...
16 blocks erased at 0x4c12: OK
16 blocks erased at 0x4c02: OK
OK
```



4.3. 板载 Kernel 编译与更新

Kernel 内核是操作系统的核心，负责管理硬件资源和系统服务。它处理进程管理、内存分配、设备驱动、文件系统等关键任务。内核在系统启动时加载，确保硬件和软件能够顺利配合。常见的开源内核如 Linux 内核，广泛应用于各种设备，从嵌入式系统到服务器。有关更多信息，请参阅内核的官方网站或社区。

4.3.1. 获取 Kernel 源码

1) 通过 github 获取源码

创建内核工作目录，然后使用如下命令下载源码：

```
$ mkdir linux;cd linux
$ git clone https://github.com/MYiR-Dev/myir-st-linux.git \
-b develop-ld25x-v6.6.48
```

2) 从光盘镜像获取源码

解压光盘镜像后将 04-Sources 下的源码压缩包 MYD-LD25X-Distribution-L6.6.48-V1.0.0.tar 解压到主机中使用。

```
$ mkdir MYD-LD25X
$ tar xf 04-Sources/MYD-LD25X-Distribution-L6.6.48-V1.0.0.tar \
-C MYD-LD25X
$ cd MYD-LD25X/MYD-LD25X-Distribution-L6.6.48-V1.0.0/MYD-LD25X-Linux-L6.6.48-V1.0.0
```

4.3.2. 编译 Kernel

1) 方法一：通过本地编译链直接编译构建

注意：该构建方式仅供调试参考，推荐使用

解压后，查看 MYD-LD25X-Linux-L6.6.48-V1.0.0 的目录结构：

```
$ ls -la
build-linux-en.sh  build-linux-zh.sh  myir-st-linux  README_enREADME_zh
```

- myir-st-linux：linux 源码，其中包含了完整的 Linux 源码
- build-linux-zh.sh：一键编译脚本，中文提示
- build-linux-en.sh：一键编译脚本，英文提示
- README-zh：中文说明文档





开始安装模块到 modules 目录

构建完成，输出目录为：/media/home/beste/01_Pro/ld25x/bsp_hu/MYD-LD25X-Linux-L6.6.48-V1.0.0/output

最后编译完成的文件会保存在和脚本同级目录的 output 中，内容如下：

```
$ ls output/ -l
total 76040
Image
Image.gz
modules.tar.gz
myb-stm32mp257x-2GB.dtb
myb-stm32mp257x-2GB-ethswitch.dtb
myb-stm32mp257x-1GB.dtb
myb-stm32mp257x-1GB-ethswitch.dtb
```

注意：这种方式编译会缺少一些关于系统功能的一些模块，这些模块是由 Yocto 依赖构建出来的。在通过这种方式调试接口或者其他功能后，建议将 Yocto 的 Linux 地址设置为本地再重新构建出完整内核及其模块。

4.3.3. 更新 Kernel

本章节描述的文件单独烧录方式为开发板上使用命令烧录，也可以使用 STM32Cube Programmer 工具在 PC 通过 TypeC 直接连接开发板烧录，STM32CubeProgrammer 工具的烧录方法请查阅《MYD-LD25X 量产指导手册-V1.2》。如下操作中 mmcblk1 为 eMMC 分区，mmcblk0 为 TF 卡分区。

1) 替换 Image 内核文件到开发板

- 替换 Image.gz 到 eMMC 开发板或者 TF 卡

将编译好的 Image.gz 拷贝到 eMMC 开发板，可以通过 scp 或者存储介质如 U 盘等传输到开发板，这里以 scp 为例，在开发板串口调试串口执行，替换前最好按照如下操作备份：

```
# cd /boot
# mv Image.gz bak-Image.gz
# scp beste@192.168.40.21:~/scpfile/Image.gz /boot
# ls /boot
bak-Image.gz
```



Image.gz

.....

TF 卡启动的系统替换 Image 基本与上述一致

2) 替换 dtb 设备树到开发板

● 替换设备树到 eMMC 开发板或者 TF 卡

将编译好的 output 目录下的 dtb 文件拷贝到 eMMC 开发板，所有设备树文件拷贝方式和下文中的操作类似，这里以 myb-stm32mp257x-2GB.dtb 为参考，使用 MYD-LD257-8E1D 的用户请在如下操作中自行修改设备树名为 1GB 的对应文件。可以通过 scp 或者存储介质如 U 盘等传输到开发板，这里以 scp 为例，在开发板串口调试串口执行，替换前最好按照如下操作备份：

```
# cd /boot
# mv myb-stm32mp257x-2GB.dtb bak-myb-stm32mp257x-2GB.dtb
# scp beste@192.168.40.21:~/scpfile/myb-stm32mp257x-2GB.dtb /boot
# ls /boot
.....
myb-stm32mp257x-2GB.dtb
bak-myb-stm32mp257x-2GB.dtb
.....
```

3) 更新 Module 到开发板

更新 modules 模块，首先将/lib/modules/6.6.48*备份到 linux-bak，然后将编译好的 modules.tar 解压到根目录就可以完成更新。

```
# cp /lib/modules/6.6.48* ~/linux-bak -rf
# rm /lib/modules/6.6.48* -rf
# tar xf modules.tar -C / > /dev/null 2>&1
```



5. 搭建基本的 Yocto 环境

5.1. 简介

Yocto 是一个开源的“umbrella”项目，意指它下面有很多个子项目，Yocto 只是把所有的项目整合在一起，同时提供一个参考构建项目 Poky，来指导开发人员如何应用这些项目，构建出嵌入式 Linux 系统。它包含 Bitbake、OpenEmbedded-Core，板级支持包，各种软件包的配置文件。

MYD-LD25X 提供了符合 Yocto 的配置文件，帮助开发者构建出可烧写在 MYD-LD25X 板上的 Linux 系统镜像。Yocto 还提供了丰富的开发文档资源，让开发者学习并定制自己的系统。由于篇幅有限，不能完整介绍 Yocto 的使用，请用户自行上网搜索。

本节适合需要对文件系统进行深度定制的开发人员，希望从 Yocto 构建出符合 MYD-LD25X 系列开发板的文件系统，同时基于它的定制化方法。初次体验使用或无特殊需要的开发人员可以直接使用 MYD-LD25X 已经提供的文件系统。

注意：构建 Yocto 不需要加载 2.3 节中的 SDK 工具链环境变量，请创建新 shell 或打开新的终端窗口。



5.2. 获取源码

我们提供两种获取源码的方式，一种是直接从米尔发布资料的 04-Sources 目录中获取压缩包，另外一种是使用 repo 获取位于 github 上实时更新的源码进行构建，请用户根据实际需要选择其中一种进行构建。由于 Yocto 构建前需要下载文件系统中所有软件包到本地，为了快速构建，MYD-LD25X 已经把相关的软件打包好，可以直接解压使用，减少重复下载的时间。米尔发布资料可以通过米尔的开发者中心获取。

5.2.1. 从发布资料获取源码压缩包

源码包在米尔开发包资料 04-Sources/MYD-LD25X-Distribution-L6.6.48-V1.0.0.tar。拷贝压缩包到用户指定目录，如解压源码包到工作目录 MYD-LD25X：

```
$ mkdir -p MYD-LD25X
$ cd MYD-LD25X
$ tar -xvf MYD-LD25X-Distribution-L6.6.48-V1.0.0.tar \
-C MYD-LD25X
$ cd MYD-LD25X/MYD-LD25X-Distribution-L6.6.48-V1.0.0/MYD-LD25X-Yocto-scarthgap-V1.0.0
```

如果您已经按照本节的内容获取到 Yocto 源码，您可以跳过下面的 5.2.2 章节。

5.2.2. 通过 github 获取

目前 MYD-LD25X 开发板的 BSP 源代码和 Yocto 源代码均使用了 github 托管并将保持长期更新，代码仓库地址请查看《MYD-LD25X SDK 发布说明-V1.2》。用户可以使用 repo 获取和同步 github 上的代码。具体操作方法如下：

```
$ mkdir -p ~/bin
$ curl https://mirrors.tuna.tsinghua.edu.cn/git/git-repo > ~/bin/repo
$ chmod a+x ~/bin/repo
$ export PATH=~/bin:${PATH}
$ export REPO_URL='https://mirrors.tuna.tsinghua.edu.cn/git/git-repo/'
$ mkdir -p MYD-LD25X-Yocto-scarthgap-V1.0.0
$ cd MYD-LD25X-Yocto-scarthgap-V1.0.0
$ repo init -u https://github.com/MYiR-Dev/myir-st-manifest.git \
--no-clone-bundle --depth=1 -m myir-stm32mp2-6.6.48-1.0.0.xml \
-b myd-ld25x-v24.12.05-scarthgap
```



```
$ repo sync
$ ls
layers
```

代码同步成功之后，同样在 MYD-LD25X 目录下得到 MYD-LD25X-Yocto-scarthgap-V1.0.0 源码一样的目录内容。

5.3. 快速编译开发板镜像

5.3.1. 执行环境变量设置脚本

在使用 Yocto 项目进行系统构建之前都需要先设置相应环境变量，所以每次在新的终端窗口编译 Yocto 时请按照如下操作执行 Yocto 源码目录下的 layers/meta-myr/scripts/envsetup.sh 文件。

使用脚本设置编译环境的操作如下：

```
$ cd MYD-LD25X-Yocto-scarthgap-V1.0.0
$ DISTRO=openstlinux-weston MACHINE=myd-ld25x source \
layers/meta-myr/scripts/envsetup.sh
```

执行配置脚本执行完成后将自动进入生成的 build-openstlinuxweston-myd-ld25x 目录。

注意：编译 yocto 需要使用普通用户，不能使用 root 用户

5.3.2. 构建镜像

米尔提供的不同镜像的区别可以查阅《MYD-LD25X SDK 发布说明-V1.2》了解。

- 构建 myir-image-full 镜像

在完成上述环境配置和 downloads 获取后，可以直接执行如下命令构建 myir-image-full 镜像：

```
$ bitbake myir-image-full
NOTE: Started PRServer with DBfile: /media/home/beste/01_Pro/ld25x/build-
openstlinuxweston-myd-ld25x/cache/prserv.sqlite3, Address: 127.0.0.1:40731, PID:
1971012
Loading cache: 100%
#####
```



```
#####| Time:
0:00:01
Loaded 4749 entries from dependency cache.
Parsing recipes: 100%
#####
#####| Time:
0:00:01
Parsing of 2997 .bb files complete (2994 cached, 3 parsed). 4754 targets, 515 skipped, 0
masked, 0 errors.
NOTE: Resolving any missing task queue dependencies

Build Configuration:
BB_VERSION          = "2.8.0"
BUILD_SYS           = "x86_64-linux"
NATIVE_SBSTRING     = "universal"
TARGET_SYS          = "aarch64-ostl-linux"
MACHINE             = "myd-ld25x"
DISTRO              = "openstlinux-weston"
DISTRO_VERSION       = "5.0.3-snapshot-20250217"
TUNE_FEATURES       = "aarch64 crc cortexa35"
TARGET_FPU          = ""
DISTRO_CODENAME      = "scarthgap"
ACCEPT_EULA_myd-ld25x = "1"
GCCVERSION          = "13.%"
.....以下省略
```

● 构建 myir-image-burn 镜像

构建 burn 镜像前需要先构建 full 镜像，因为 burn 镜像需要将 full 镜像的文件系统、内核文件等拷贝到 burn 镜像的系统中用于烧录。

myir-image-burn 镜像分为 myir-image-burn-1g 和 myir-image-burn-2g，分别适用于 MYD-LD257-8E1D 和 MYD-LD257-8E2D 两个型号，burn 镜像用于烧录到 TF 卡，然后通过 TF 卡启动后将 TF 卡系统内的 myir-image-full 镜像烧录到 eMMC 的一种便于量产化的镜像包，该镜像的具体操作烧录办法请查阅《MYD-LD25X 量产指导手册-V1.2》，这里按照开发板型号不同选择执行以下其中一种命令后即可构建 burn 镜像：

➤ MYD-LD257-8E1D

```
$ bitbake myir-image-burn-1g
```



➤ MYD-LD257-8E2D

```
$ bitbake myir-image-burn-2g
```

构建完成之后，编译出来的镜像在“tmp-glibc/deploy/images/myd-ld25x/”。

生成的镜像文件按照《MYD-LD25X 量产指导手册-V1.2》通过 STM32CubeProgrammer 更新即可。

● 构建 raw 文件用于 TF 卡烧录

如果需要将镜像烧录到 TF 卡，那么需要通过 sdcard-raw-tools 工具中的脚本，首先需要输入以下命令构建 sdcard-raw-tools：

```
$ bitbake sdcard-raw-tools
```

然后就可以进入以下目录：

```
$ cd tmp-glibc/deploy/images/myd-ld25x/scripts/
```

进入后运行脚本可以看到如何使用：

```
$ ./create_sdcard_from_flashlayout.sh
```

```
[ERROR]: bad number of parameters
```

Help:

```
./create_sdcard_from_flashlayout.sh [-h|--help] [--compress] <FlashLayout file>
```

```
-h      :      this help
```

```
--help:      this help
```

```
--compress:      compress the raw image generated
```

```
--force-rootfs: force to use predefined rootfs size (3906 MB)
```

By setting SDCARD_SIZE on shell environment or calling the script with it you can limit the size of RAW sdcard

```
SDCARD_SIZE=<value on MB>
```

```
ex.: SDCARD_SIZE=2048 ./script/create_sdcard_from_flashlayout.sh <flashlayout>
```

this exemple limit the size of sdcard to 2GB (2048MB)

By setting DEVICE on shell environment or calling the script with it you can customize the command

```
ex.: DEVICE=sdb ./script/create_sdcard_from_flashlayout.sh <flashlayout>
```



接下来将以 burn 镜像为例介绍如何使用该脚本输出 raw.xz 的压缩包文件，raw 文件用于烧录到 TF 卡中，这里输出 xz 的压缩包文件方便传输。

输入以下命令开始制作适用于 MYD-LD257-8E2D 的 burn 镜像的 raw.xz 文件，full 镜像类似，只需要选择不同的 Flashlayout 文件即可，如果需要制作 MYD-LD25X-8E1D 的 burn 镜像 raw.xz 文件，如下操作中高亮的 tsv 文件选择对应目录下的 1GB 文件即可，这里不再赘述。

```
$ ./create_sdcard_from_flashlayout.sh --compress \
../flashlayout_myr-image-burn-2g/optee/FlashLayout_sdcard_myb-stm32mp257x-
2GB-optee.tsv
Create Raw empty image: ../flashlayout_myr-image-burn-
2g/optee/../../FlashLayout_sdcard_myb-stm32mp257x-2GB-optee.raw of 5442MB
Create partition table:
[CREATED] part 01: fsbla1 [partition size 256.0 KiB]
[CREATED] part 02: fsbla2 [partition size 256.0 KiB]
[CREATED] part 03: metadata1 [partition size 256.0 KiB]
[CREATED] part 04: metadata2 [partition size 256.0 KiB]
[CREATED] part 05: fip-a [partition size 4.0 MiB]
[CREATED] part 06: fip-b [partition size 4.0 MiB]
[CREATED] part 07: u-boot-env [partition size 512.0 KiB]
[CREATED] part 08: bootfs [partition size 64.0 MiB]
[CREATED] part 09: vendorfs [partition size 183.0 MiB]
[CREATED] part 10: rootfs [partition size 4.0 GiB]
[CREATED] part 11: userfs [partition size 1.1 GiB]
.....
```

需要注意的是，上述选择的 tsv 文件需要是 sdcard 的，而不是 eMMC 的。等待完成后，raw.xz 文件就会输出在上层目录也就是 tmp-glibc/deploy/images/myd-ld25x/目录下了。

```
$ ls ../*.raw.xz
-rw- 444893344 ../FlashLayout_sdcard_myb-stm32mp257x-2GB-optee.raw.xz
```

解压该文件后，得到的 raw 文件就可以按照《MYD-LD25X 量产指导手册-V1.2》中的 Win32DiskImager 烧录章节进行烧录了。



5.4. 构建交叉编译链

Yocto 提供可构建出交叉编译工具链功能，用于底层或上层应用开发者使用的工具链和相关的头文件或库文件，免去用户手动制作或编译依赖库。用于编译 U-Boot 和 linux 内核代码，附带目标系统的头文件和库文件，方便应用开发者移植应用在目标设备上。下面讲解如何构建工具链。

本节只简单对米尔提供的交叉编译工具链做构建说明，使用如下命令生成交叉编译工具链：

```
$ bitbake -c populate_sdk myir-image-full
```

等待构建完成后，将在“*tmp-glibc/deploy/sdk/*”路径下生成交叉编译工具链，安装方法请查看 2.2 节。

```
myir-image-full-openstlinux-weston-myd-ld25x.rootfs-x86_64-toolchain-5.0.3-snapshot.sh
```

构建包含 Qt 的编译链需要先修改 Yocto 目录的 myir-image-full 文件在执行构建工具链命令：

```
$ cd layers/meta-myir/meta-myir-openstlinux/recipes-st/images/  
$ vi myir-image-full.bb  
SUMMARY = "OpenSTLinux weston image with basic Wayland support (if enable in  
distro)."  
LICENSE = "Proprietary"  
  
include recipes-st/images/st-image.inc  
  
inherit core-image features_check  
#inherit populate_sdk populate_sdk_qt5
```

上述文件中高亮的注释内容要取消后回到 build 目录执行构建工具链命令。



6. 如何适配底板硬件

6.1. meta-bsp 层介绍

Yocto 项目的“层模型”是一种用于嵌入式和物联网 Linux 创建的开发模型，它将 Yocto 项目与其他简单的构建系统区别开来。层模型同时支持协作和定制。层是包含相关指令集的存储库，这些指令集告诉 OpenEmbedded 构建系统应该做什么。

meta-myr 层基于在 ST 官方的 meta-st 层建立的适用于 MYD-LD25X 开发板的层，其下面的 meta-bsp 层中包含 BSP、GUI、发行版配置、中间件或应用程序的各种元数据和配方。用户可以在这个“层模型”的基础上适配基于 MYD-LD25X 开发板设计的硬件，定制自己的应用，从而构建适合自己的系统镜像，本节主要介绍 meta-myr 层，其包含的具体内容如下：

```
$ tree -a -L 1 layers/meta-myr/meta-myr-stm32mp
```

```
layers/meta-myr/meta-myr-stm32mp
```

```
|— classes  
|— CODE_OF_CONDUCT.md  
|— conf  
|— License.md  
|— README -> README.md  
|— README.md  
|— recipes-bsp  
|— recipes-connectivity  
|— recipes-core  
|— recipes-devtools  
|— recipes-extended  
|— recipes-graphics  
|— recipes-kernel  
|— recipes-myr  
|— recipes-security  
|— recipes-st  
|— recipes-support
```

部分层说明：



表 6-1. meta-myr 层内容说明

源代码与数据	描述
conf	包含当前层路径信息和机器软件配置
recipes-bsp	包含 atf、uboot 以及固件等配置信息
recipes-kernel	包含 linux 内核的资源与第三方固件资源
recipes-myr	包含文件系统的米尔自定义包配置信息

在进行系统移植时，需要重点关注的是负责硬件初始化和系统引导的 recipes-bsp 部分，负责 Linux 系统的内核和驱动实现的 recipes-kernel 部分。

6.2. 如何创建自己的 machine

在开发过程中，用户有时需要创建自定义板配置。本节将通过一个实例讲解用户如何创建属于自己的 machine。

6.2.1. 在 Yocto 创建一个板子配置

1) 选择类似 machine 文件

复制一个类似的 machine 文件，并重命名为一个你板子的指定名字，和 MYD-LD25X 类似的 machine 文件在 `layers/meta-myr/meta-myr-stm32mp/conf/machine` 这个目录，进入这个目录，machine 文件如下：

```
$ cd layers/meta-myr/meta-myr-stm32mp/conf/machine
$ ls
include  myd-ld25x.conf
```

2) 复制并重命名

找到类似的 machine 文件后，复制并重命名为你自己的 machine 文件，如：

```
$ cp myd-ld25x.conf test-myd-ld25x.conf
$ ls
myd-ld25x.conf test-myd-ld25x.conf
```

3) MACHINE 配置文件

在 MACHINE 配置文件中需要注意的一些参数配置：

```
KERNEL_DEVICETREE += "myir/myb-stm32mp257x-1GB.dtb \
myir/myb-stm32mp257x-2GB.dtb \
```



```
myir/myb-stm32mp257x-2GB-ethswitch.dtb \  
myir/myb-stm32mp257x-1GB-ethswitch.dtb \  
"
```

这个是指定包含的内核的设备树，这里指定了 myb-ld25x 全部类型的设备树，有哪些设备树可以查阅 4.1 章节的介绍，不过这里指定的是最后编译完成的 dtb 文件，而不是 dts 文件。

4) 编译并测试

当 machine 文件创建完成之后，您可以通过编译并下载测试，执行如下命令编译最小镜像测试，并且注意需要修改配置命令的 MACHINE 为自定义的 MACHINE 名,同时也注意 flash 型号类型：

```
$ cd MYD-LD25X-Yocto-scarthgap-V1.0.0  
$ DISTRO=openstlinux-weston MACHINE=test-myd-ld25x source layers/meta-  
myir/scripts/envsetup.sh  
$ bitbake core-image-minimal
```

编译完成后生成的镜像 **build-openstlinuxweston-test-myd-ld25x/tmp-glibc/deploy/images/test-myd-ld25x/**，拷贝并按照《MYD-LD25X 量产指导手册-V1.2》通过 STM32CubeProgrammer 烧录到开发板中，启动测试：

```
root@test-myd-ld25x:~#
```

6.2.2. 在 uboot 创建板子配置文件

在开发过程中，用户一般需要根据自己的板子要求创建属于自己的板子配置文件，本小节将通过一个简单实例演示如何一步步创建自己的板子配置文件。

1) 创建 board

在创建自己的 board 配置时，用户可以在对应的板子目录通过复制并重命名建立自己板子配置文件，一般板子配置文件都在 uboot 源码的 board 目录。进入进入 uboot 源码 board 子目录下的 myir 子目录，复制 myd_ld25x 文件夹为 test_ld25x，如下所示：

```
$ cd MYD-LD25X-Uboot-L2023.10-V1.0.0/board/myir/  
$ cp myd_ld25x/ test_ld25x -rf  
$ ls  
myd_ld25x  test_ld25x
```



进入 test_ld25x 目录，修改 myd-ld25x.c 为 test-ld25x.c;修改 Makefile，将 myd-ld25x.o 修改为 test_ld25x.o;

```
$ mv myd-ld25x.c test-ld25x.c
$ vi Makefile
$ cat Makefile
# SPDX-License-Identifier: GPL-2.0

obj-y += test-ld25x.o
```

2) 制作板子.config 文件

- 修改新创建的板子 Kconfig

进入 test_ld25x 目录，修改 Kconfig 文件为如下红色标记一致：

```
if ld25x

config SYS_BOARD
    default "test_ld25x"

config SYS_VENDOR
    default "myir"

config SYS_CONFIG_NAME
    default "test_ld25x"

endif
```

接着在 arch/arm/mach-stm32mp/Kconfig.25x 文件增加如下红色标记内容（这个会影响 make menuconfig 配置）：

```
source "board/st/stm32mp2/Kconfig"
source "board/myir/test_ld25x/Kconfig"
```

- 创建新板子的头文件

从源码根目录下进入 include/configs 目录，复制 myd_ld25x.h 为 test_ld25x.h，如下：



```
$ cd include/configs
$ cp myd_ld25x.h test_ld25x.h
$ ls -l test_ld25x.h
-rw-rw-r-- 1 myir myir 1620 Sep  5 11:49 test_ld25x.h
```

注意：由于 SYS_CONFIG_NAME 的名字改为了 test_ld25x.h，所以此头文件改为 test_ld25x.h。

- 定制系统板子配置文件

从源码根目录下进入 configs 目录，根据 Flash 的型号选择复制不同的 defconfig 为 test-defconfig，这里选择 2GB 为例：

```
$ cd configs
$ cp myd_ld25x_2G_defconfig test_ld25x_2G_defconfig
$ ls test_ld25x_2G_defconfig -la
-rw-rw-r-- 1 beste beste 1963 8 月  1 15:23 test_ld25x_2G_defconfig
```

经过这几个步骤基本上已经是定制好了板子，如果用户需要用 yocto 编译，那么还需要根据以下操作修改在 layers 中 uboot 的 defconfig 配置信息：

```
$ vi layers/meta-myir/meta-myir-stm32mp/recipes-bsp/u-boot/u-boot-stm32mp-config.inc
...
UBOOT_CONFIG[default_stm32mp25] ?= "test_ld25x_2G_defconfig,u-boot.dtb"
...
```

3) 编译

将修改后的 uboot 提交 git 后，需要通过 Yocto 编译请参考 4.2.2 章节的内容。



6.3. 如何创建您的设备树

6.3.1. 板载设备树层级的介绍

设备树是一种数据结构，它通过特有的语法格式描述片上片外的设备信息。如将部分设备信息结构存放到 device tree 文件中。uboot 或内核最终将其 device tree 编译成 dtb 文件，使用过程中通过解析该 dtb 来获取板级设备信息。uboot 的 dtb 和 kernel 中的 dtb 是一致的。

1) myir-st-uboot 设备树介绍

下面 MYD-LD25X 的 uboot 各部分设备树的信息列表，以便用户参考：

表 6-2. MYD-LD25X U-Boot 设备树列表

项目	设备树	描述
U-Boot	arch/arm/dts/myb-stm32mp257x-2GB.dts	2GB RAM 的 U-Boot 设备树定义文件
	arch/arm/dts/myb-stm32mp257x-2GB-resmem.dtsi	2GB RAM 的保留内存设备树配置包含文件
	arch/arm/dts/myb-stm32mp257x-2GB-u-boot.dtsi	U-Boot 特定设备树配置包含文件
	arch/arm/dts/myb-stm32mp257x-1GB.dts	1GB RAM 的 U-Boot 设备树定义文件
	arch/arm/dts/myb-stm32mp257x-1GB-resmem.dtsi	1GB RAM 的保留内存设备树配置包含文件
	arch/arm/dts/myb-stm32mp257x-1GB-u-boot.dtsi	U-Boot 特定设备树配置包含文件
	arch/arm/dts/myb-stm32mp257x-base.dtsi	基础设备树配置包含文件

编译 MYD-LD25X 的 uboot 源码时会合并相关的所有 dts/dtsi 文件，生成 uboot 阶段默认使用的 myb-stm32mp257x-2GB.dtb 或者 myb-stm32mp257x-1GB.dtb。

2) myir-st-linux 设备树介绍

myir-st-linux 的设备树层级关系为：myb-stm32mp257x-base.dtsi+myb-stm32mp257x-ethswitch.dtsi+myb-stm32mp257x-2GB-resmem.dtsi -> myb-stm32mp257x-2GB-ethswitch.dts 和 myb-stm32mp257x-2GB.dts，1GB 的设备树类似。下面是 MYD-LD25X 的各部分设备树的详细信息列表，以便用户开发参考：



表 6-3. MYD-LD25X Linux 设备树列表

项目	设备树	描述
Kernel	arch/arm64/boot/dts/myir/myb-stm32mp257x-2GB.dts	2GB RAM 的设备树定义文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-2GB-ethswitch.dts	2GB RAM 的以太网交换机的设备树定义文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-2GB-resmem.dtsi	2GB RAM 的保留内存设备树包含文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-1GB.dts	1GB RAM 的设备树定义文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-1GB-ethswitch.dts	1GB RAM 的以太网交换机的设备树定义文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-1GB-resmem.dtsi	1GB RAM 的保留内存设备树包含文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-base.dtsi	基础设备树配置包含文件
	arch/arm64/boot/dts/myir/myb-stm32mp257x-ethswitch.dtsi	以太网交换机设备树配置包含文件

6.3.2. 设备树的添加

1) Uboot 创建设备树

- 选择类似的设备树文件

从源码根目录下进入 `arch/arm/dts` 目录，复制 `myb-stm32mp257x-2GB.dts` 为 `test-stm32mp257x-2GB.dts`，1GB 配置类似，选择 1GB 对应的设备树即可，2GB 参考如下：

```
$ cp myb-stm32mp257x-2GB.dts test-stm32mp257x-2GB.dts
```

- 修改设备树 Makefile

接着修改该目录下的设备树 Makefile，增加内容如下：

```
...
dtb-$(CONFIG_STM32MP25X) += \
    stm32mp257f-dk.dtb \
    test-stm32mp257x-2GB.dts \
...
```



- 修改板子配置文件

然后进入源码目录下的 configs 目录，修改 test_ld25x_2G_defconfig 配置文件默认的设备树，如下：

```
...  
CONFIG_ENV_SECT_SIZE=0x80000  
CONFIG_DEFAULT_DEVICE_TREE="test-stm32mp257x-2GB"  
...
```

2) Kernel 创建设备树

MYIR 系列设备树在 *arch/arm64/boot/dts/myir* 目录，用户在该目录也可以通过复制 MYIR 的设备树然后重命名，方法过程如 uboot 类似，请参考以上步骤。



6.4. 如何根据您的硬件配置 CPU 功能管脚

实现一个功能引脚的控制是一个较为复杂的系统开发过程之一，其中包含了引脚的配置，驱动的开发，应用的实现等等步骤，本节将以实例来讲解功能管脚的控制实现。

6.4.1. GPIO 管脚配置的方法

MYD-LD25X 板的 IO 管脚大部分定义在 arch/arm64/boot/dts/myir/myb-stm32mp257x-base.dtsi 设备树文件中。由于 st 官方已经把每一个 IO 管脚可能使用到的 IOMUX 功能，以及与此功能相对应的 IOPAD 属性都已经准备好，所以用户在配置的时候只需要直接在数组中包括我们想使用的 IO 管脚及功能的宏就可以了。

1) 查看 pinctrl 配置规则

st pinctrl 配置格式：`#define STM32_PINMUX(port, line, mode) (((PIN_NO(port, line)) << 8) | (mode))`

其中：

- Port: 表示 GPIO 引脚所在的端口，如引脚 PD11 中的字符'D'。
- Line: GPIO 引脚在其端口中的编号。每个端口有多个引脚，从 0 开始编号，如 PD11 引脚中的 11。
- Mode: 引脚工作模式的模式值

关于模式值，在文件 include/dt-bindings/pinctrl/stm32-pinfunc.h 中找这个参数的定义如下：

```
/* define PIN modes */
#define GPIO      0x0
#define AF0       0x1
.....
#define AF13      0xe
#define AF14      0xf
#define AF15      0x10
#define ANALOG    0x11
#define RSVD      0x12
```

2) 在设备树上配置 gpio



下面将使用 DTS 文件来进行设备硬件资源的申请及分配，用户可以在 myb-ld25xdx-base.dts 文件下操作 DTS，定义 led 设备节点如下：

```
gpio_leds_test {  
    compatible = "gpio-leds";  
  
    led0 {  
        label = "LED-Blue";  
        gpios = <&gpioz 5 GPIO_ACTIVE_HIGH>;  
        default-state = "on";  
        linux,default-trigger = "heartbeat";  
    };  
};
```



6.5. 如何使用自己配置的管脚

我们在 U-Boot 或 Kernel 的设备树中配置后的管脚，可以在相应 U-Boot 或 Kernel 中进行使用，从而实现对管脚的控制。

6.5.1. U-Boot 中使用 GPIO 管脚

1) uboot 终端命令控制

uboot 可以直接使用命令来控制 GPIO 的设置。如开发板上的 User 灯，它的引脚号是 PH4，它的管脚号为 $(72-65) * 16 + 4 = 116$ ，公式为 $PIN_NO('H',4)=(0x48-0x41)*0x10+4=(72-65)*16+4=116$ 。

注：72 为 H 的 ASCLL 值，65 为 A 的 ASCLL 值。

```
STM32MP> gpio clear 116
gpio: pin 116 (gpio 116) value is 0
STM32MP> gpio set 116
gpio: pin 116 (gpio 116) value is 1
```

可以用直接看到对应的开发板上 D8 的 User 灯亮起和关闭。

2) gpio 函数控制

用户可以在 uboot 阶段使用 GPIO 值控制，在代码中实现 GPIO 值控制的功能，如下列 phy 的电源复位控制。

- 在 uboot 使用复位引脚

在 uboot 的 board/myir/myd-ld25x/myd-ld25x.c 使用该引脚,如下:

```
static void led_gpio(void)
{
    unsigned int gpio_led=116;
    gpio_request(gpio_led, "led-gpio");
    gpio_direction_output(gpio_led, 0);
    gpio_set_value(gpio_led, 0);
    gpio_free(gpio_led);
}
```



7. 应用的开发与启动

Linux 应用的移植通常分为两个阶段，开发调试阶段和生产部署阶段。开发调试阶段我们可以使用米尔构建好的 SDK 对我们编写好的应用进行交叉编译然后远程拷贝到目标主机进行测试。生产部署阶段需要为应用编写配方文件，并使用 Bitbake 构建生产镜像。

7.1. 基于 Makefile 的应用

Makefile 其实就是一个文档，里面定义了一系列的编译规则，它记录了原始代码如何编译的详细信息！ Makefile 一旦写好，只需要一个 make 命令，整个工程完全自动编译，极大的提高了软件开发的效率。在开发 Linux 程序时，不管是内核，驱动，应用，Makefile 得到了普遍的应用。

make 是一个命令工具，是一个解释 Makefile 中指令的命令工具。它可以简化编译过程里面所下达的指令，当执行 make 时，make 会在当前的目录下搜寻 Makefile (or makefile) 这个文本文件，执行对应的操作。make 会自动的判别原始文件是否经过了变动，从而自动重新编译更改的源代码。

下列将以一个实际的示例（在 MYD-LD25X 开发板上实现按键控制）来讲述 Makefile 的编写与 make 的执行过程。Makefile 有其自身的一套规则。

target ... : prerequisites ...

command

- target 可以是一个 object file(目标文件)，也可以是一个执行文件，还可以是一个标签 (label)。
- prerequisites 就是要生成那个 target 所需要的文件或是目标。
- command 也就是 make 需要执行的命令。

```
TARGET = $(notdir $(CURDIR))
objs := $(patsubst %c, %o, $(shell ls *.c))
$(TARGET)_test:$(objs)
                $(CC) -o $@ $^
%.o:%.c
                $(CC) -c -o $@ $<
clean:
```



```
rm -f $(TARGET)_test *.all *.o
```

部分参数说明：

- \$(CURDIR): 表示 Makfile 当前目录全路径
- \$(notdir \$(path)): 表示把 path 目录去掉路径名, 只留当前目录名, 比如当前 Makefile 目录为/home/myir/MYD-LD25X/key_led, 执行为就变为 TARGET = key_led
- \$(patsubst pattern, replacement,text): 用 replacement 替换 text 中符合格式 "pattern" 的字符, 如\$(patsubst %c, %o, \$(shell ls *.c)), 表示先列出当前目录后缀为.c 的文件, 然后换成后缀为.o
- CC: C 编译器的名称
- CXX: C++编译器的名称
- clean: 是一个约定的目标

Key 实现代码如下：

```
#include <linux/input.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

/* ./key_led /dev/input/event1 noblock */
int main(int argc, char **argv)
{
    int fd,bg_fd;
    int err, len, i;
    unsigned char flag;
    unsigned int data[1];
    char *bg = "/sys/class/leds/LED-Blue/brightness";
```



```
struct input_event event;

system("echo none > /sys/class/leds/LED-Blue/trigger");

if (argc < 2)
{
    printf("Usage: %s <dev> [noblock]\n", argv[0]);
    return -1;
}

if (argc == 3 && !strcmp(argv[2], "noblock"))
{
    fd = open(argv[1], O_RDWR | O_NONBLOCK);
}
else
{
    fd = open(argv[1], O_RDWR);
}
if (fd < 0)
{
    printf("open %s err\n", argv[1]);
    return -1;
}

while (1)
{
    len = read(fd, &event, sizeof(event));
    if (event.type == EV_KEY)
    {
        if (event.value == 1)//key down and up
        {

            printf("key test \n");
            bg_fd = open(bg, O_RDWR);
            if (bg_fd < 0)
```



```
        {  
            printf("open %d err\n", bg_fd);  
            return -1;  
        }  
        read(bg_fd,&flag,1);  
        if(flag == '0')  
            system("echo 1 > /sys/class/leds/LED-  
Blue/brightness"); //led on  
        else  
            system("echo 0 > /sys/class/leds/LED-  
Blue/brightness");//led off  
        }  
    }  
}  
return 0;  
}
```

使用 `make` 命令进行编译并生成目标机器上的可执行文件 `target_bin`。

加载 SDK 环境变量到当前 shell:

```
$ source ~/ld25x_toolchain/environment-setup-aarch64-poky-linux
```

执行 `make`:

```
$ make
```

从上一个命令的结果可以看到，使用的编译器是通过设置脚本中定义的 `CC` 变量建立的编译器。将 `key_led_test` 可执行文件通过网络（`scp` 等），u 盘等传输介质拷贝到开发板的 `/usr/sbin` 目录下，执行以下命令，然后按 `USER` 按键，可以看到蓝灯亮灭的情况：

```
# key_led_test /dev/input/event0  
key test  
key test  
key test  
key test
```



7.2. 基于 Qt 的应用

Qt 是一个跨平台的图形应用开发框架，被应用在不同尺寸设备和平台上，同时提供不同版权版本供用户选择。MYD-LD25X 使用 Qt 5.15.13 版本进行应用开发。在 Qt 应用开发中，推荐使用 QtCreator 集成开发环境，可以在 Linux PC 下开发 Qt 应用，自动化地交叉编译为开发板的 ARM 架构程序。详情参考《MYD-LD25X QT 应用笔记-V1.1》。



7.3. 应用程序开机自启动

1) 开机自启动服务

在开发板系统中，默认会包含一个 autorun 的服务，这个服务会在开机运行时自动执行/usr/bin/autorun.sh 这个脚本，autorun.service 位于/lib/systemd/system/autorun.service。

执行如下命令查看 autorun 服务状态：

```
# systemctl status autorun
```

```
● autorun.service - auto run hmi after weston
```

```
Loaded: loaded (/lib/systemd/system/autorun.service; enabled; preset: enabled)
```

```
Active: active (running) since Fri 2023-03-03 10:08:52 UTC; 542ms ago
```

```
Process: 3912 ExecStart=/usr/bin/autorun.sh (code=exited, status=0/SUCCESS)
```

```
Main PID: 3919 (mxapp2)
```

```
Tasks: 6 (limit: 2032)
```

```
Memory: 17.5M
```

```
CGroup: /system.slice/autorun.service
```

```
└─3919 /usr/sbin/mxapp2
```

```
3 月 03 10:08:53 myd-ld25x autorun.sh[3919]: libpng warning: iCCP: known incorrect sRGB profile
```

```
3 月 03 10:08:53 myd-ld25x autorun.sh[3919]: libpng warning: iCCP: known incorrect sRGB profile
```

如果需要添加开机启动就执行的操作命令，可以将命令添加到/usr/bin/autorun.sh 中，借助 autorun 服务实现：

```
# vi /usr/bin/autorun.sh
```

```
source /etc/profile.d/weston_profile.sh
```

```
source /etc/profile.d/pulse_profile.sh
```

```
/usr/sbin/mxapp2 &
```

```
echo "This is a test log" > /dev/ttySTM0
```

```
exit 0
```

上述操作将一条测试打印输出到调试串口，在重启后可以观察到。



注意：autorun 服务中的打印均会保留在 systemctl status 中，只有执行这个命令才可以观察到，除非在配置中将打印重定向到/dev/ttySTM0 调试串口中。

重新运行验证：

```
# systemctl restart autorun
This is a test log
```

2) 开机自启动应用

如果需要自己的应用在 Yocto 中配置并安装到文件系统中，并且实现开机自动启动，我们可以参考 meta-myrir/meta-myrir-stm32mp/recipes-myrir/autorun 的写法编写一个配方 (recipes)。

进入 autorun 包的目录：

```
$ cd sources/meta-myrir/meta-myrir-stm32mp/recipes-myrir/autorun
$ ls
autorun  autorun.bb
```

能够看到 autorun 目录下由两部分组成，一个是 bb 文件，一个 autorun 的资源目录，这个目录下存放着 autorun.bb 文件中会使用到的所有文件。

以下操作将会参考这个 package 来创建一个新的 test-app 的 package 并放到 myir-image-full 镜像中，并且实现开机自动运行。

拷贝 autorun 并更改名称为 test-app:

```
$ cd meta-myrir/recipes-myrir
$ cp autorun test-app -rf
$ ls
... autorun test-app ...
```

将 test-app 目录下的文件名称都更改为 test-app:

```
$ cd test-app
$ mv autorun.bb test-app.bb
$ mv autorun/ test-app/
$ cd test-app
$ mv autorun.service test-app.service
```

在 test-app 中，原本的 autorun 程序使用于自动运行的应用程序，这里我们将其更换为一个简单的测试 shell 脚本，并且内容如下：

```
$ ls
```



```
licenses autorun test-app.service
$ rm autorun
$ vi test-app.sh      #创建一个脚本并添加以下内容，完成后保存关闭
#!/bin/sh
echo "====This is a test shell script====" > /dev/ttyS0
```

这个脚本用于将这个提示信息打印到调试串口中，然后将这个脚本添加到 service 服务中，让开机后运行 test-app.service 服务时能够运行这个脚本，脚本假设位于文件系统的/usr/bin/目录下。

```
$ vi test-app.service      #打开服务并文件修改为和如下内容一致
[Unit]
Description=auto run hmi after weston
After=weston-graphical-session.service

[Service]
#Type=oneshot
Type=forking
#ExecCondition=/usr/bin/pgrep -x weston
ExecStart=/usr/bin/test-app.sh

[Install]
WantedBy=multi-user.target
```

修改完成后跳转上层目录修改 bb 文件，打开 bb 文件并将其修改为和下文中一致，建议直接拷贝：

```
$ cd ../
$ vi test-app.bb
SUMMARY = "Test app"
DESCRIPTION = "test application"
LICENSE = "GPL-2"
LIC_FILES_CHKSUM = "file://licenses/GPL-2;md5=94d55d512a9ba36caa9b7df079bae19f"
S = "${WORKDIR}"
SRC_URI = "\
```



```
file://licenses/GPL-2 \  
file://test-app.sh \  
file://test-app.service \  
"  
  
inherit systemd  
  
do_install() {  
    install -d -m 755 ${D}/etc  
    install -d -m 755 ${D}${systemd_system_unitdir}  
  
    install ${WORKDIR}/test-app.sh ${D}/usr/bin/  
    install -m 644 ${WORKDIR}/test-app.service  
    ${D}${systemd_system_unitdir}/test-app.service  
}  
  
SYSTEMD_PACKAGES = "${PN}"  
SYSTEMD_SERVICE_${PN} = "test-app.service"  
SYSTEMD_AUTO_ENABLE_hmi = "enable"  
  
FILES_${PN} += "/"
```

保存后，test-app 这个 package 就创建完成了，然后将 test-app 添加到 myir-image-full，打开 full 镜像的 bb 文件并将 test-app 添加进去：

```
$ vi meta-myr/recipes-images/images/myir-image-full.bb  
....  
IMAGE_INSTALL_append = "  
    test-app \  
"
```

添加完成后保存并构建 full 镜像。

```
$ bitbake myir-image-full
```

然后将构建好的镜像更新到开发板，启动即可观察到。



8. 参考资料

- Linux kernel 开源社区

<https://www.kernel.org/>

- Yoto 项目 BSP 开发指南

<https://www.yoctoproject.org/docs/3.1.1/bsp-guide/bsp-guide.html>

- Yocto 项目 Linux 内核开发手册

<https://www.yoctoproject.org/docs/3.1.1/kernel-dev/kernel-dev.html>

- Yocto 开发指导

<https://www.yoctoproject.org/>



附录一 联系我们

深圳总部

地址：深圳市龙岗区坂田街道发达路云里智能园 2 栋 6 楼 04 室

负责区域：广东、广西、海南、重庆、云南、贵州、四川、西藏、香港、澳门

传真：0755-25532724

电话：0755-25622735

武汉研发中心

地址：武汉东湖新技术开发区关南园一路 20 号当代科技园 4 号楼 1601 号

电话：027-59621648

华东地区

地址：上海市浦东新区金吉路 778 号浦发江程广场 1 号楼 805 室

负责区域：上海、福建、浙江、江苏、安徽、山东

传真：021-62087085

电话：021-62087019

华北地区

地址：北京市大兴区荣华中路 8 号院力宝广场 10 号楼 901 室

负责区域：辽宁、吉林、黑龙江、北京、天津、河北、山西、内蒙古、湖北、湖南、江西、河南、陕西、甘肃、宁夏、青海、新疆

传真：010-64125474

电话：010-84675491

销售联系方式

网址：www.myr.cn

邮箱：sales.cn@myir.cn

技术支持联系方式

邮箱：support.cn@myir.cn

武汉研发中心电话：027-59621648

深圳总部技术电话：0755-22316235



如果您通过邮件获取帮助时，请使用以下格式书写邮件标题：

[公司名称/个人--开发板型号] 问题概述

这样可以使我们更快速跟进您的问题，以便相应开发组可以处理您的问题。



附录二 售后服务与技术支持

凡是通过米尔电子直接购买或经米尔电子授权的正规代理商处购买的米尔电子全系列产

品，均可享受以下权益：

- 1、6 个月免费保修服务周期
- 2、终身免费技术支持服务
- 3、终身维修服务
- 4、免费享有所购买产品配套的软件升级服务
- 5、免费享有所购买产品配套的软件源代码，以及米尔电子开发的部分软件源代码
- 6、可直接从米尔电子购买主要芯片样品，简单、方便、快速；免去从代理商处购买时，漫长的等待周期
- 7、自购买之日起，即成为米尔电子永久客户，享有再次购买米尔电子任何一款软硬件产品的优惠政策
- 8、OEM/ODM 服务

如有以下情况之一，则不享有免费保修服务：

- 1、超过免费保修服务周期
- 2、无产品序列号或无产品有效购买单据
- 3、进液、受潮、发霉或腐蚀
- 4、受撞击、挤压、摔落、刮伤等非产品本身质量问题引起的故障和损坏
- 5、擅自改造硬件、错误上电、错误操作造成的故障和损坏
- 6、由不可抗拒自然因素引起的故障和损坏

产品返修：

用户在使用过程中由于产品故障、损坏或其他异常现象，在寄回维修之前，请先致电米尔电子客服部，与工程师进行沟通以确认问题，避免故障判断错误造成不必要的运费损失及周期的耽误。

维修周期：

收到返修产品后，我们将即日安排工程师进行检测，我们将在最短的时间内维修或更换并寄回。一般的故障维修周期为 3 个工作日（自我司收到物品之日起，不计运输过程时间），由于特殊故障导致无法短期内维修的产品，我们会与用户另行沟通并确认维修周期。

维修费用：

在免费保修期内的产品，由于产品质量问题引起的故障，不收任何维修费用；不属于免费保修范围内的故障或损坏，在检测确认问题后，我们将与客户沟通并确认维修费用，我们仅收取元器件



材料费，不收取维修服务费；超过保修期限的产品，根据实际损坏的程度来确定收取的元器件材料费和维修服务费。

运输费用：

产品正常保修时，用户寄回的运费由用户承担，维修后寄回给用户的费用由我司承担。非正常保修产品来回运费均由用户承担。

